Docker Refresher: Containerize the Cars CRUD App

Goal: Package the existing FastAPI + plain HTML + SQLite application into **one Docker image**, run it as **one container**, and publish the image to **Docker Hub**.

Before starting: You should already know what images and containers are. This is a short, practical follow-up, not a full introduction to Docker. Work from the root of the existing `cars-crud/` project.

1. Quick recap

Term	Meaning
`Dockerfile`	Instructions for building an image.
Image	Packaged application code, Python, and dependencies.
Container	A running instance of an image.
Registry	Online service for storing and distributing images.
Docker Hub	A popular registry at https://hub.docker.com/.
Tag	Image version label, such as `cars-crud:v1`.
Volume	Storage that can survive removing a container.

Prerequisite: Install Docker Desktop (Windows/macOS) or Docker Engine (Linux), and make sure the Docker service is running. Check:

```
docker --version
docker info
```

2. Pulling Docker images faster in China (Azure proxy)

Docker Hub can sometimes be slow to access from China. For this course, an **Azure Container Registry proxy** is available at **`efreiwut.azurecr.io`**. It provides an alternative route for pulling supported public images through Microsoft Azure.

For example, a normal Docker Hub pull is:

```
docker pull python:3.12
```

When working locally, you can instead pull the same Python image and tag through the proxy:

```
docker pull efreiwut.azurecr.io/dockerhub/library/python:3.12
```

The tag still matters: `:3.12` and `:3.12-slim` refer to **different images**. For a direct `FROM` instruction, the equivalent choices are:

```
# Standard Docker Hub base image
FROM python:3.12
```

```
# Same image tag through the course's Azure proxy
FROM efreiwut.azurecr.io/dockerhub/library/python:3.12
```

For this Cars CRUD app, we use the smaller `3.12-slim` variant, so the corresponding proxy path is `efreiwut.azurecr.io/dockerhub/library/python:3.12-slim`.

Where you build	Recommended image source
Your computer, with normal Docker Hub access	`python:3.12-slim`
Your computer, if Docker Hub is slow	`efreiwut.azurecr.io/dockerhub/library/python:3.12-slim`
GitHub Actions runner	The ordinary `python:3.12-slim` base image normally works; the Azure proxy may also be used if accessible.

Important distinctions:

- The Azure address is a **pull-through path for supported cached images**, not the Docker Hub destination for publishing your Cars CRUD image. You will still push `YOUR_DOCKER_ID/cars-crud:v1` to **your own Docker Hub repository**.
- You do not need to use the proxy in GitHub Actions just because you use it locally. Both paths are valid wherever the configured registries are reachable.
- In production, teams often pull from and push to **their own container registries**, using image names such as `registry.example.com/team/cars-crud:v1`. The registry hostname is part of the image reference.
- If the Azure proxy returns an authorization or image-not-found error, check your course's access instructions, the `/cache/library/` path, and the image tag. The proxy must be configured to serve the requested image.

The Dockerfile below uses a **build argument** so you can switch between Docker Hub and the proxy without editing the file each time.

3. Create a Dockerfile

Create a file named exactly **`Dockerfile`** (no extension) at the project root, next to `requirements.txt`:

```
cars-crud/
├── Dockerfile          # Add this
├── .dockerignore       # Add in the next section
├── requirements.txt
├── app/
│   ├── main.py
│   ├── database.py
│   └── static/          # HTML, CSS, JavaScript
└── tests/
```

Copy this **complete Dockerfile**:

```
ARG BASE_IMAGE=python:3.12-slim
FROM ${BASE_IMAGE}

WORKDIR /app

COPY requirements.txt ./
RUN python -m pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/

RUN mkdir -p /data && chown 10001:10001 /data
USER 10001:10001
ENV CARS_DB_PATH=/data/cars.db

EXPOSE 8000

CMD ["python", "-m", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--
port", "8000"]
```

What each Dockerfile instruction does

Instruction	In our application
`ARG`	Declares the base image reference used when building (default: Docker Hub).
`FROM`	Starts with the selected Python 3.12 slim base image.
`WORKDIR`	Sets `/app` as the working directory inside the image.
`COPY`	Copies Python dependencies first, then the `app/` code and HTML files.
`RUN`	Executes commands **during image building** (install packages, create `/data`).
`USER`	Runs the application as an unprivileged numeric user instead of root.
`ENV`	Sets the SQLite path to `/data/cars.db` inside the container.
`EXPOSE`	Documents that the app listens on port 8000; it **does not** publish the port.

Instruction	In our application
`CMD`	Specifies the command executed **when the container starts**.

Notice two details:

- The server binds to `0.0.0.0` **inside** the container, so the published port can reach it. This differs from using `127.0.0.1` inside a container.
- We copy `requirements.txt` before the source code so Docker can reuse the dependency-install layer when only application code changes. We install only application dependencies, not the testing tools.

Other useful instructions to recognize: `ENTRYPOINT` (main executable) and `HEALTHCHECK` (container health checks). You do not need them for this project.

4. Create `.dockerignore`

Create `.dockerignore` beside the Dockerfile:

```
.git/
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
tests/
*.db
*.db-*
*.sqlite*
.env
```

This keeps Git history, local Python environments, test files, and local SQLite databases out of the Docker build context. **Never bake your local `cars.db`, passwords, or access tokens into an image.**

5. Build and run the container

Run from the **project root** (the final `.` means "use this directory as the build context"):

```
docker build -t cars-crud:local .
```

If Docker Hub is slow on your **local computer in China**, use the course's Azure proxy **for this build only**:

```
docker build --build-arg
BASE_IMAGE=efreiwut.azurecr.io/dockerhub/library/python:3.12-slim -t cars-
crud:local .
```

The build argument replaces the Dockerfile's default `BASE_IMAGE` for this invocation. **GitHub Actions can keep the default `python:3.12-slim`** when it builds the committed Dockerfile; no editing or extra build argument is required there. The resulting application is packaged in the same way.

Start one container with a persistent named volume for SQLite:

```
docker run -d --name cars-app -p 127.0.0.1:8000:8000 --mount
type=volume,source=cars-data,target=/data cars-crud:local
```

Open:

- **Application:** http://localhost:8000/
- **API documentation:** http://localhost:8000/docs

Create a car using the HTML page. Its JavaScript calls the FastAPI endpoints; both are served by **the same container**. No separate frontend server or database container is needed.

Understand the `docker run` flags:

Flag	Meaning
`-d`	Run in the background.
`--name cars-app`	Give the container an easy-to-use name.
`-p 127.0.0.1:8000:8000`	Connect local port 8000 to the container's port 8000, accessible from this computer.
`--mount type=volume,source=cars-data,target=/data`	Save the SQLite database in a named Docker volume.
`cars-crud:local`	Image to start.

The `CARS_DB_PATH` value in the Dockerfile points at `/data/cars.db`. Without the volume, deleting the container would also delete its SQLite data. The volume is kept separately from the container.

Verify persistence:

```
docker stop cars-app
docker rm cars-app
docker run -d --name cars-app -p 127.0.0.1:8000:8000 --mount
type=volume,source=cars-data,target=/data cars-crud:local
```

Refresh the application. Previously created cars should still appear.

6. Docker CLI cheat sheet

Command	Purpose
`docker build -t cars-crud:local .`	Build an image from the local Dockerfile.
`docker image ls`	List local images.
`docker ps`	List running containers.
`docker ps -a`	List running and stopped containers.
`docker logs cars-app`	View container output and errors.
`docker stop cars-app`	Stop a running container.
`docker start cars-app`	Restart an existing stopped container.
`docker rm cars-app`	Remove a **stopped** container, not its named volume.
`docker volume ls`	List named Docker volumes.
`docker exec -it cars-app sh`	Open a shell inside the running container.
`docker tag SOURCE TARGET`	Give an image another name/tag.
`docker push IMAGE`	Upload an image to a registry.
`docker pull IMAGE`	Download an image from a registry.

To follow application logs live, run `docker logs -f cars-app` and stop following with Ctrl+C. This does **not** stop the running container.

7. Create a Docker Hub account and repository

1. Go to https://hub.docker.com/ and select **Sign up**. Create a Docker account and verify it. Record your **Docker ID** (username).
2. Sign in and open **My Hub > Repositories > Create repository**.
3. Choose your Docker ID as the namespace and name the repository `cars-crud`.
4. Choose **Public** for an easily shareable training image, or **Private** if you want restricted access. Create the repository.
5. Under **Account settings > Personal access tokens**, generate a token with **Repo Read & Write** permission and an appropriate expiration. Copy and store it securely when shown. You will need it again for the GitHub Actions exercise.

A Docker Hub repository (`YOUR_DOCKER_ID/cars-crud`) stores **container images**, whereas your GitHub repository stores **source code**.

Authenticate from your terminal

```
docker login --username YOUR_DOCKER_ID
```

When prompted for the password, paste the **Docker Hub personal access token**, not your GitHub password. Never paste that token into a Dockerfile, Git commit, or screenshot.

Tag and push the image

Replace `YOUR_DOCKER_ID` with your Docker Hub username in each command:

```
docker tag cars-crud:local YOUR_DOCKER_ID/cars-crud:v1
docker push YOUR_DOCKER_ID/cars-crud:v1
```

The name follows `NAMESPACE/REPOSITORY:TAG`. Open your Docker Hub repository and verify that the **`v1`** tag appears.

Optional: you can also tag the same image `latest`, but remember that `latest` is only a **tag name**; it does not automatically find the newest version.

8. Pull your published image

On a computer with Docker installed (or after removing the local tagged image), you can download it:

```
docker pull YOUR_DOCKER_ID/cars-crud:v1
```

You can run it using the same `docker run` options from Section 5, replacing `cars-crud:local` with `YOUR_DOCKER_ID/cars-crud:v1`. If your repository is private, the computer pulling the image must have permission to access it.

9. Common problems

Problem	What to check
`Cannot connect to the Docker daemon`	Start Docker Desktop or the Docker service.
`failed to read Dockerfile`	Run `docker build` from the directory containing `Dockerfile`.
Python module not found	Check that `requirements.txt` and `app/` are present in the build context.
Browser cannot connect	Check `docker ps`, `docker logs cars-app`, and the `-p` mapping.
Port 8000 already in use	Stop the local Uvicorn process or change the host port (for example, `127.0.0.1:8001:8000`).
`Permission denied` writing SQLite	Use the named volume mounted at `/data`; custom bind mounts may need ownership adjustments for user `10001`.
Docker Hub push denied	Check the namespace, repository name, token permissions, and `docker login`.

Problem	What to check
Azure proxy pull fails	Confirm the image name/tag and `efreiwut.azurecr.io/dockerhub/library/` path; check proxy availability or course access.

Important: This is a no-authentication training app. Keep its port bound to `127.0.0.1` while learning; do not expose it publicly without adding access controls.

Next: DOCKER_PUBLISHING_EXERCISE.md uses the Dockerfile and Docker Hub repository from this guide.

References

- Dockerfile reference
- Docker Python guide
- Create a Docker Hub account
- Create a Docker Hub repository
- Docker Hub personal access tokens